Applied Machine Learning

Practice Set with Step-by-Step Solutions

Scenario-based problems · BITS WILP — SE ZG568 / SS ZG568

How to use this set

Each problem is a realistic scenario you might face in industry — a team has built something, observed something, or needs to decide something. Read the scenario, sketch your own answer on paper, and then read the worked solution. The goal is not memorisation; it is recognising the underlying concept when it shows up wearing different clothes.

Topics covered:

- Decision Trees — entropy, information gain, traps
- Bagging & Out-of-Bag estimation — the correlation floor
- Random Forest — mtry, feature importance, correlated columns
- PCA — scale, leakage, one-hot data
- K-Means — initialisation, outliers, drift
- DBSCAN — variable density, high-dimensions, MinPts traps
- Perceptron & MLP — XOR, forward pass, depth vs width
- Activations & vanishing gradient — sigmoid chains, dying ReLU
- Regularisation — L1 vs L2 sparsity, dropout, early stopping
- CNN — output-size & parameter computations, transfer learning
- RNN, LSTM & Attention — forgetting, gates, QKV mechanics
- Fairness & Interpretability — parity, equalised odds, SHAP, LIME

Topic 1: Decision Trees

Problem 1.1 — The high-cardinality trap

A clinical team builds a decision tree on hospital admission data with 200 features. After training, the tree's most important feature, accounting for 60% of the impurity reduction, is patient_id. A junior analyst is excited and announces: “patient_id is the strongest predictor of readmission.”

(a) Why is patient_id getting picked first by the algorithm despite carrying no real predictive signal?

(b) What does this say about the relationship between information gain and feature usefulness?

(c) Name two concrete practices that prevent this trap before training.

Solution — Problem 1.1

(a) Why patient_id wins

Information gain measures how much a split reduces entropy. patient_id is unique per row — each value appears in exactly one example. So splitting on patient_id produces leaves with one row each, all pure (entropy = 0). The IG is therefore maximal. This is mathematically correct yet practically useless: a leaf containing one row generalises to nothing.

(b) IG ⁄ usefulness

IG only measures TRAINING entropy reduction. It is blind to whether the split GENERALISES. Any feature with very high cardinality (IDs, raw timestamps, free-text columns) can game IG.

(c) Practical fixes

- Drop identifier columns (patient_id, transaction_id) before training.
- Use IG-ratio (C4.5) or Gini with a penalty for high-cardinality splits — divides IG by the “split info” of the feature.
- Set a sensible min_samples_leaf (e.g., 20) so the algorithm cannot make a per-row leaf in the first place.

Bottom line: patient_id is the “best” split by IG but the worst by generalisation. Always inspect tree-imported features for cardinality red flags, drop IDs, and use leaf-size constraints.

Problem 1.2 — When domain experts disagree with the algorithm

A senior loan officer hands you a 10-feature dataset and tells you: “Split first on credit_score — that is what we always look at.” Your ID3 algorithm picks employment_status as the root, with higher information gain. The loan officer is unhappy.

(a) Walk through how the algorithm’s decision is mathematically defensible — what does it imply about the relationship between employment_status and the default label in YOUR training data?

(b) When SHOULD domain knowledge override information gain?

(c) How would you reconcile the two views in your final report?

Solution — Problem 1.2

(a) What the algorithm is saying

Higher IG on employment_status means: in your specific training set, employment_status partitions defaults / non-defaults more cleanly than credit_score. Perhaps employment_status correlates with default risk more directly than the score (which is a derived, smoothed quantity).

(b) When to override

- If the training data is too small or biased toward a subset of customers, the algorithm's ranking may be noisy.
- If there is a known regulatory expectation to weight a feature explicitly (e.g., the FICO score).
- If the "highest IG" feature would create unfair / illegal disparate impact across protected groups (e.g., ZIP code as a proxy for race).

(c) Reconciliation

> **Bottom line:** Show the loan officer BOTH trees — root on employment_status (your algorithm's pick) and a constrained tree that splits first on credit_score (their request). Compare validation accuracy, fairness metrics, and interpretability. Use the comparison to justify a final choice — domain expertise and algorithmic evidence working together, not against.

Problem 1.3 — Pre-pruning vs post-pruning under a deadline

You have one week to deliver a tree-based model on a 500k-row regression problem. Your two realistic options are: (a) train a deep unpruned tree and post-prune via cross-validation, (b) pre-prune from the start with max_depth = 6 and min_samples_leaf = 50. Each has trade-offs.

(i) For each option, list (1) compute cost, (2) tuning complexity, (3) likely quality.

(ii) Recommend one approach, justified by the deadline constraint.

Solution — Problem 1.3

Option A — Train deep, post-prune

- Compute: highest — you train the full deep tree, then run cost-complexity pruning with cross-validation.
- Tuning: complex — you must search over the cost-complexity parameter α.
- Quality: typically the best in the long run; the algorithm finds the sweet spot empirically.

Option B — Pre-prune with fixed hyper-parameters

- Compute: cheap — you stop early.
- Tuning: simple — only 2 hyper-parameters to set.
- Quality: typically slightly worse than well-tuned post-pruning but very close, and far more robust to bad hyper-parameter choices.

Recommendation

> **Bottom line:** Pre-prune. With a one-week deadline, you want a model that ships and a tuning problem that fits in the time budget. Pre-pruning with reasonable defaults (depth 6, min-leaf 50) typically gets you 95% of the way to optimal post-pruned quality at 30% of the engineering time. Reserve post-pruning for the next iteration.

Topic 2: Bagging & Out-of-Bag Estimation

Problem 2.1 — The correlation floor

A research team has built 200 bagged decision trees for predicting energy load. The variance of a single tree's prediction on the validation set is $\sigma^2 = 0.5$. Theory says that with INDEPENDENT trees, the bagged variance should be $\sigma^2 / 200 \approx 0.0025$. Yet they measure the bagged variance to be 0.305.

(a) Use the formula $\text{Var(bag)} = \rho\cdot\sigma^2 + (1-\rho)\cdot\sigma^2/M$ to estimate the implied correlation ρ between trees.

(b) Why is the correlation so high in their setup, and what TWO concrete changes would drive it down?

(c) After their interventions, ρ drops to 0.1. Compute the new bagged variance and the relative improvement.

Solution — Problem 2.1

(a) Implied ρ

$0.305 = \rho\cdot 0.5 + (1-\rho)\cdot 0.5/200 = 0.5\rho + 0.0025\cdot(1-\rho)$.

$0.305 = 0.5\rho + 0.0025 - 0.0025\rho$

$0.3025 = 0.4975\rho \Rightarrow \rho \approx 0.608$.

So the trees are about 60% correlated — a typical figure when bagging deep trees on tabular data without further randomisation.

(b) Why so correlated, and how to reduce

- All trees share the same dominant feature at the root (e.g., one very predictive variable), so their early splits agree. Fix: switch to a Random Forest (random feature subset of size $\sqrt{p}$ at each split).
- All trees train on overlapping bootstrap samples; with n large, the overlap is essentially the full dataset. Fix: train with a smaller subsample-per-tree (e.g., 50% rows), or use Extra Trees with random thresholds at each split.

(c) After ρ = 0.1

$\text{Var(bag)} = 0.1\cdot 0.5 + 0.9\cdot 0.5/200 = 0.050 + 0.00225 = 0.05225$.

> **Bottom line:** Variance dropped from 0.305 to 0.052 — a ~6× reduction, achieved not by adding more trees but by decorrelating them. The lesson: once $\rho > 0.5$, more trees do almost nothing; you need a different mechanism (feature randomness, smaller subsamples).

Problem 2.2 — OOB vs cross-validation, under a tight data budget

A team has only 10,000 labelled examples. They are debating two evaluation strategies:

- (A) 80–20 train/test split + 5-fold cross-validation inside the train set.
- (B) Use the full 10,000 for training and rely on the bagging out-of-bag (OOB) error.

Compare on three dimensions: (i) effective training set size, (ii) reliability of the error estimate, (iii) total compute cost. Recommend one.

Solution — Problem 2.2

(i) Effective training set

- A: 8,000 training examples per fold (because 20% is reserved for the held-out test, then 5-fold inside the 8,000).
- B: 10,000 — every example contributes.

(ii) Reliability

- A: CV gives 5 fold-wise estimates ± std. Test-set estimate is a single number from 2,000 examples — small but unbiased.

- B: OOB error averages across all trees and all OOB examples — very low variance, slightly optimistic because of mild leakage between bootstrap samples (each tree sees ~63%).

(iii) Compute cost

- A: train 5 × (model size) for CV + 1 final model — roughly 6× compute.
- B: train ONCE (M bagged trees) — same compute as a single bagging run.

Recommendation

> **Bottom line:** Use OOB for fast iteration during development (B), then validate the FINAL model on a small permanent hold-out set you set aside on day 1. This combines OOB's zero-overhead estimate with the bias-resistance of an honest test set. Avoid full 5-fold CV unless you specifically need fold-level variance — it costs 6× compute for marginal information beyond OOB.

Problem 2.3 — When bagging fails to help

Your team wraps Bagging around a logistic regression baseline (instead of around decision trees). To their surprise, the bagged version is no better than a single logistic regression.

(a) Why doesn't bagging help here? What property of logistic regression makes it bagging-resistant?

(b) Give two examples of base learners where bagging genuinely helps, and explain what they have in common.

Solution — Problem 2.3

(a) Why bagging fails on logistic regression

Bagging reduces VARIANCE. Logistic regression is a HIGH-BIAS, LOW-VARIANCE model — its decision boundary is a fixed hyperplane, and small data perturbations produce nearly identical coefficients. Averaging hyperplanes that are already almost identical gives you the same hyperplane. Bagging therefore has nothing to reduce.

(b) Where bagging works

- Deep, unpruned decision trees: tiny data changes produce wildly different trees — high variance, low bias. Bagging eats the variance, keeps the (good) bias.
- Neural networks with many parameters and aggressive training: similarly high variance from random initialisation and SGD stochasticity. "Snapshot ensembles" bag the same network at multiple training checkpoints.

> **Bottom line:** Bagging's sweet spot is high-variance learners. If your base learner is already stable and biased, you need BOOSTING (which targets bias), not bagging.

Topic 3: Random Forest

Problem 3.1 — A dominant feature swamps the forest

In your RF model for fraud detection, the feature account_age dominates: nearly every tree splits on it at the root. Variance reduction across the ensemble is disappointing — adding more trees barely helps.

(a) Refer to $\text{Var(ensemble)} = \rho\cdot\sigma^2 + (1-\rho)\cdot\sigma^2/M$. What is happening to ρ in this setup?

(b) What single hyper-parameter would you change, and how?

(c) After the change, how would you verify that tree diversity has improved? Name one quantitative check.

Solution — Problem 3.1

(a) Why ρ is high

All trees converge to the same dominant root split → similar internal structure → highly correlated predictions → ρ close to 1. Adding more trees gives essentially Var(ensemble) ≈ $\rho \cdot \sigma^2$, which barely depends on M.

(b) Hyper-parameter to change

Decrease max_features (mtry). Default for classification is √p; force it lower — e.g., mtry = $\log_2(p)$ or even a small constant. Now at each split, account_age is only IN THE RANDOM SUBSET part of the time. Trees are forced to find OTHER informative features. Diversity goes up; ρ goes down.

(c) Verification

> **Bottom line:** Compute the average pairwise correlation between tree predictions on a held-out batch. Before the change: ρ ≈ 0.7. After: aim for ρ ≈ 0.2. You can also check root-split feature usage — a healthy RF should use 5–10 different features at the root across its trees, not just one.

Problem 3.2 — Feature importance with correlated columns

Your RF on weather data reports these Gini-based importances:

- temperature: 0.30
- dewpoint: 0.05
- humidity: 0.10
- wind_speed: 0.20

temperature and dewpoint are well-known to be highly correlated. A junior wants to drop dewpoint because "its importance is < 0.1 — clearly useless".

(a) What is the failure mode of Gini importance with correlated features?

(b) What would permutation importance reveal that the default importance hides?

(c) State your final recommendation.

Solution — Problem 3.2

(a) Why Gini hides correlated features

When two features are correlated, the tree splits on whichever the random subset contained first. The credit for the impurity reduction goes to the WINNER, and the loser gets zero credit at that split — even though it could have done the same job. Gini importance therefore SPLITS impurity credit between correlated features, making both look weaker than either really is.

(b) Permutation importance

Permutation importance shuffles ONE feature at a time and measures the drop in accuracy. If temperature and dewpoint are perfectly correlated, shuffling EITHER would only cause a small drop (the other carries the signal). Shuffling BOTH would cause a large drop. So permutation importance reveals the COLLECTIVE redundancy — both features carry similar information.

(c) Recommendation

> **Bottom line:** Do NOT drop dewpoint on the basis of low Gini importance alone. Either (i) run permutation importance to see what each feature is worth on its own and jointly, or (ii) keep both features — RF is robust to redundancy, and removing dewpoint costs nothing if it is redundant but hurts you if you misread the importance.

Problem 3.3 — Picking n_estimators under a latency budget

You are deploying RF in a real-time fraud-detection system with a strict 50ms per-prediction budget. Each tree's prediction takes 0.4ms. Your team wants the variance benefit to be within 1% of the "infinite-tree" limit.

Assume Var(ensemble) = $\rho\cdot\sigma^2 + (1-\rho)\cdot\sigma^2/M$ with realistic $\rho = 0.2$.

(a) Find the smallest M such that the second term $(1-\rho)\cdot\sigma^2/M$ is within 1% of the first term (the asymptotic floor).

(b) Check your latency budget: how many trees can you afford in 50ms?

(c) Recommend a final n_estimators with a brief justification.

Solution — Problem 3.3

(a) M for 1% of the asymptotic variance

Want $(1-\rho)\cdot\sigma^2 / M \le 0.01 \cdot (\rho\cdot\sigma^2) \Rightarrow (1-\rho) / M \le 0.01\cdot\rho \Rightarrow M \ge (1-\rho) / (0.01\cdot\rho)$.

With $\rho = 0.2$: $M \ge 0.8 / (0.01 \cdot 0.2) = 0.8 / 0.002 = 400$ trees.

(b) Latency budget

50ms / 0.4ms per tree = 125 trees max.

(c) Recommendation

> **Bottom line:** 125 trees. Beyond ~125, you simply do not have the latency. With 125 trees, $(1-\rho)/M\cdot\sigma^2 = (0.8/125)\cdot\sigma^2 = 0.0064\cdot\sigma^2$, compared to floor $\rho\cdot\sigma^2 = 0.2\cdot\sigma^2$. So you are within ~3% of the floor — not quite 1%, but very close. The TRUE leverage now is to drive ρ down (with smaller mtry), not to add more trees.

Topic 4: Principal Component Analysis

Problem 4.1 — A satellite-imaging scale catastrophe

A satellite imagery team applies PCA on a dataset with two features per measurement: surface temperature in Kelvin (mean 300, sample variance ≈ 400) and barometric pressure in Pascals (mean 1.0×10^5, sample variance ≈ 10^6). They forget to standardise.

(a) Compute the ratio of the two variances and explain which axis PC1 will essentially align with.

(b) After z-score standardisation, what becomes of each feature's variance?

(c) The team observes that after standardisation, PC1 captures only 55% of variance — much less than the 99%+ they got without standardisation. Are they getting WORSE results? Explain.

Solution — Problem 4.1

(a) Variance ratio

ratio = 10^6 / 400 = 2,500. Pressure variance is 2,500× larger than temperature variance — so PC1 is essentially the PRESSURE axis. Temperature contributes negligibly to PC1.

(b) After z-scoring

$z_i = (x_i - \mu) / \sigma$. Each feature now has mean 0 and variance 1. Neither dominates by virtue of its measurement scale.

(c) Why 55% is BETTER, not worse

Bottom line: Pre-standardisation, PC1 captured 99%+ because it absorbed all of pressure's artificial-scale variance. That number was a measurement-unit artifact — change Pascals to kilopascals and PC1's share would plummet. Post-standardisation, the 55% reflects the TRUE structural correlation between temperature and pressure. PC1 is now interpretable as a real combined axis, not a unit-of-measurement choice.

Problem 4.2 — PCA on a mostly one-hot dataset

You apply PCA on a customer dataset where 90% of features are one-hot encoded categorical variables (region, plan_type, payment_method, etc.). PC1 explains 8% of variance, PC2 explains 6%, then a slow tail. The team is disappointed.

(a) Why does PCA struggle on heavily one-hot encoded data?

(b) What does the slow "no-elbow" variance profile usually tell you about the underlying data structure?

(c) Suggest TWO alternative dimensionality-reduction strategies and when each is appropriate.

Solution — Problem 4.2

(a) The fundamental mismatch

One-hot encoding turns ONE categorical variable into K binary columns (one per category). Each column is mostly zeros with a sparse one. Variances are tiny ($p \cdot (1-p)$ for category proportion p) and there is no linear correlation structure between the binary columns of the same category (in fact they are anti-correlated by construction). PCA looks for LINEAR axes of variance — there is just not much linear structure to find.

(b) "No elbow" = spread structure

A slow, gradual decline in explained variance means the information is distributed roughly evenly across many directions. There is no compact representation — you need almost as many PCs as features for any reasonable variance threshold. PCA is barely doing anything.

(c) Alternatives

- Target encoding / mean encoding: replace each category with the mean target value of training examples in that category. Collapses K one-hot columns into 1 informative numerical column.
- Embedding layers via a small neural network or matrix factorisation (e.g., entity embeddings) — learn a dense, low-dimensional representation for each category from the actual prediction task.

Bottom line: PCA is not designed for sparse one-hot data. Either re-encode categoricals to dense numerical features (target encoding) or learn task-aware embeddings.

Problem 4.3 — A subtle data leak through PCA

A team builds the following pipeline:

- Step 1: fit PCA on the FULL dataset (train + future test rows already collected).
- Step 2: split 80–20 into train and test.
- Step 3: train a classifier on the train PCA-features.

Validation accuracy is 95%. After release, real production accuracy is 75%.

(a) Identify the data leak. Be specific about which step caused it.

(b) Why does the leak inflate validation accuracy?

(c) Write the CORRECT pipeline in three steps.

Solution — Problem 4.3

(a) Where the leak is

Step 1. PCA learns its rotation (the eigenvectors) using statistics computed from BOTH train and test rows. So the projection axes are tuned to maximise variance over the test rows too — the test set is no longer independent of the model.

(b) Why this inflates accuracy

Test-set rows have been seen during PCA fitting. The classifier's downstream features for those test rows are slightly optimised for them. The held-out metric over-estimates real-world performance, because in production you will receive rows that PCA has never seen.

(c) Correct pipeline

- Step 1: split 80–20 first.
- Step 2: fit PCA on the TRAIN ROWS ONLY. Save the fitted PCA object.
- Step 3: TRANSFORM both train and test using the same fitted PCA, train the classifier on transformed train, evaluate on transformed test. In production, transform new rows with the same fitted PCA.

Bottom line: Any preprocessing that LEARNS something from the data (PCA, scaling means/std, target encoding, even imputation) must be fit on TRAIN ONLY. Otherwise your test set is no longer a real test set.

Topic 5: K-Means Clustering

Problem 5.1 — One bad point destroys a cluster

A grocery chain uses K-Means with K=5 on customer (latitude, longitude) addresses to plan store locations. One customer accidentally entered their address as (0, 0) lat/long (point in the Atlantic Ocean). The team is surprised that ONE of the cluster centroids has shifted noticeably southward.

(a) Why does ONE faraway point pull a centroid? Refer to the K-Means update rule.

(b) Why does median-based clustering (K-Medoids) handle this better?

(c) Suggest TWO practical defences against this kind of data-entry outlier.

Solution — Problem 5.1

(a) Why the mean is pulled

The K-Means update step recomputes each centroid as the ARITHMETIC MEAN of points assigned to it. The mean is highly sensitive to outliers — a single point at (0, 0) with the others around (40, −74) pulls the centroid toward (0, 0) proportionally to its weight. With n_cluster = 100 honest customers + 1 outlier, the outlier already shifts the mean by 1%. Closer outliers shift it more.

(b) K-Medoids

K-Medoids replaces the mean with the MEDIAN-like medoid (the actual data point that minimises distance sum). The medoid is robust — a single extreme point cannot move it unless half the cluster is also extreme.

(c) Practical defences

- Pre-process: clip or remove geographic outliers using domain bounds (lat/long must be valid for the country).

- Use K-Medoids (or its scalable cousin, CLARA / PAM) instead of K-Means whenever the application is reasonably small to medium scale (≤ 10^5 points).

Bottom line: K-Means is the wrong tool when even a single bad point can corrupt a centroid. Validate inputs, or switch to K-Medoids for robustness.

Problem 5.2 — Different results every run

A team runs vanilla K-Means with K=4 on a dataset that visibly has 4 clean clusters. They re-run 5 times with different random seeds and get noticeably different results: cluster boundaries shift, sizes change, sometimes one cluster splits while another merges. The dataset has not changed.

(a) What is the root cause of the non-determinism?

(b) Walk through what specifically happens in a "bad" initialisation.

(c) Name the single, well-known modification that dramatically reduces this variance, and state ONE theoretical guarantee it gives.

Solution — Problem 5.2

(a) Root cause

Vanilla K-Means picks the initial K centroids UNIFORMLY AT RANDOM from the data. The choice of initial centroids strongly influences which local minimum the algorithm converges to. Bad initialisations → bad local minima → different runs find different solutions.

(b) A bad initialisation

Suppose the 4 true clusters are at (0,0), (10,0), (0,10), (10,10), and the random init places 3 of the 4 centroids near (0,0) and one near (10,10). On the first assignment, almost every point gets pulled to one of the 3 nearby centroids; the (10,10) centroid grabs only its own cluster. Iterations cannot recover — once a centroid is stuck inside a cluster, it cannot leave to find a different one. K-Means converges to a poor 4-cluster solution that misses two of the true clusters.

(c) K-Means++

Bottom line: K-Means++ picks the FIRST centroid uniformly at random, but each subsequent centroid is picked with probability proportional to the SQUARED distance to the nearest already-chosen centroid. Far-away points are quadratically more likely to be picked, so centroids start well-spread. Theoretical guarantee: in expectation, K-Means++ initialisation is within O(log K) of the optimal clustering — vanilla random init has no such bound.

Problem 5.3 — Concept drift after deployment

A retail company clustered 100,000 customers using K-Means with K=4 in January. Two months later, the team notices that customers in cluster 2 are now behaving very differently from when the clusters were built (a new product launched and changed everyone's purchase patterns).

(a) What general issue is at play here — what is it called?

(b) Should the company simply re-run K-Means? What are TWO operational risks of frequent reclustering?

(c) Propose a practical monitoring protocol — what signal should trigger a recluster?

Solution — Problem 5.3

(a) The general issue

CONCEPT DRIFT (specifically, "data drift" or "covariate shift"): the underlying customer behaviour distribution has changed since the clustering was fit. The cluster definitions are now stale — they describe last quarter, not the customer base today.

(b) Risks of frequent reclustering

- Cluster identity changes: customer X might be in "cluster 2" one quarter and "cluster 4" the next. Downstream business systems that reference cluster IDs (marketing campaigns, dashboards) break.
- Apparent improvement may be just noise: K-Means is stochastic, and re-running on slightly drifted data could give superficially different clusters even when nothing material changed. You risk over-reacting to noise.

(c) Monitoring protocol

Bottom line: Track the AVERAGE within-cluster distance over time. As drift accumulates, points sit farther from their assigned centroid. Also track the percentage of points that would JUMP clusters if you re-ran assignment with the current centroids. When either signal crosses a pre-set threshold (e.g., average distance 2× the original, or > 15% of points would jump), trigger a controlled recluster with clear cluster-ID mapping rules. Avoid reclustering on a fixed time schedule.

Topic 6: DBSCAN

Problem 6.1 — Why everything became noise

You apply DBSCAN with ε = 0.1 and MinPts = 10 on a 50-dimensional gene-expression dataset. The result: 100% of points are labelled NOISE. The team is confused — visually the data appears to have structure when projected to 2-D with t-SNE.

(a) What property of high-dimensional spaces broke DBSCAN in this setup?

(b) Why does ε = 0.1 effectively become "practically infinite" or "practically zero" at 50 dimensions?

(c) Recommend ONE practical alternative pipeline.

Solution — Problem 6.1

(a) Curse of dimensionality

In high dimensions, pairwise Euclidean distances concentrate: the gap between the nearest and farthest neighbour shrinks as a fraction of the average distance. Everything is roughly equidistant from everything else. DBSCAN's ε-ball notion of "close" loses meaning.

(b) Numerical reality

If features are in [0, 1] each, the diagonal of the unit hypercube in 50-D is $\sqrt{50} \approx 7.07$. Average pairwise distance is on the order of $\sqrt{(d/6)} \approx \sqrt{8} \approx 2.83$. An ε = 0.1 ball is microscopic relative to the data spread — almost no point has 10 neighbours, so almost no point becomes core, so almost everything is labelled noise.

(c) Pipeline

Bottom line: Reduce dimensionality FIRST (PCA to 5–10 dims, or UMAP if non-linear structure is suspected), then run DBSCAN in the low-dimensional space. Alternative: switch entirely to HDBSCAN, which uses a HIERARCHY of density thresholds and is more robust to the high-D collapse than vanilla DBSCAN.

Problem 6.2 — Variable density across the dataset

A team applies DBSCAN with ε = 50 metres, MinPts = 5 to GPS trace data for a city. Result:

- Downtown: a single giant cluster (thousands of merged clusters that should have been separate).
- Suburbs: almost every point is labelled noise.

(a) Explain why a single ε cannot serve both regions.

(b) What happens if they SHRINK ε to fix downtown? What happens if they GROW ε to fix the suburbs?

(c) Name two algorithms that solve this fundamental issue.

Solution — Problem 6.2

(a) Why one ε fails

DBSCAN treats ε as a global density threshold. Downtown points have many neighbours within 50 metres — they all become core, and DBSCAN's recursive density-reachability links them into one supercluster across district boundaries. Suburban points have few neighbours within 50 metres — they fail the MinPts threshold and are labelled noise.

(b) The dilemma

- Shrink ε (say, to 10m): downtown clusters split sensibly, but every suburban point becomes noise.
- Grow ε (say, to 200m): suburbs cluster, but downtown becomes one mega-cluster covering the entire city centre.

There is no ε that works for both.

(c) Algorithms that handle variable density

> **Bottom line:** HDBSCAN (Hierarchical DBSCAN) constructs a hierarchy of density levels and extracts the most stable clusters — works at multiple density scales simultaneously. OPTICS (Ordering Points To Identify Cluster Structure) produces a reachability plot revealing nested cluster structure across densities. Both are drop-in upgrades for this scenario.

Problem 6.3 — MinPts = 1 "to be safe"

A junior on your team sets MinPts = 1 in DBSCAN "to make sure every point becomes core". After running, they are confused: each point is its own cluster (n clusters for n points).

(a) Walk through, step by step, why MinPts = 1 produces n clusters.

(b) Why is MinPts = 1 a meaningless setting semantically?

(c) What is the standard rule of thumb for MinPts on a d-dimensional dataset, and why?

Solution — Problem 6.3

(a) Mechanism

DBSCAN's core-point criterion: a point p is core if its ε-neighbourhood (including itself) has ≥ MinPts points. With MinPts = 1, EVERY point satisfies this — it has at least itself. So every point is core. Then density-reachability requires that a core point reaches another core point within ε. If ε is small, no point reaches another, so each starts its own cluster, of size one. (Even with large ε, you end up with one giant cluster of all points — also useless.)

(b) Why semantically meaningless

DBSCAN's whole purpose is to distinguish DENSE regions from NOISE. MinPts = 1 abolishes the noise category. There is no longer a distinction between "genuine cluster" and "lonely point" — every point qualifies as core, so density-based clustering has lost its information.

(c) Rule of thumb

> **Bottom line:** MinPts ≥ d + 1 for d-dimensional data; in practice MinPts = 2·d is common. For 2-D, MinPts = 4 is the typical default. The rationale: a meaningful cluster should have enough density that even its boundary points have a non-trivial neighbourhood. MinPts = 1 (or 2) leaks too much, and too-high MinPts (say, 100) makes everything noise.

Topic 7: Perceptron and MLP

Problem 7.1 — The XOR ceiling

A junior wants to learn the XOR function with a single perceptron. After 5,000 epochs of training, accuracy plateaus at 75%. They are convinced "the algorithm is broken".

(a) Geometrically, why can a single perceptron never exceed 75% on XOR?

(b) What is the smallest neural-network architecture that DOES solve XOR? Sketch the layer sizes.

(c) Provide concrete weights for a 2-2-1 MLP with ReLU activations that solves XOR exactly, and verify on input (1, 0).

Solution — Problem 7.1

(a) The geometric ceiling

A perceptron computes y = step(w·x + b), which is a LINEAR DECISION BOUNDARY. In 2-D, that is a single straight line. XOR's positive class is {(0,1), (1,0)} and negative class is {(0,0), (1,1)}. The two classes are arranged diagonally — no single straight line can separate them. The best a line can do is separate 3 of the 4 points correctly → 75%.

(b) Smallest sufficient architecture

A 2-input, 2-hidden-unit, 1-output MLP. Two hidden units give two "half-planes"; the second layer combines them. This is the canonical XOR demonstration network.

(c) Worked example with ReLU

Weights: $W_1 = [[1, 1], [1, 1]]$, $b_1 = [0, -1]$, $W_2 = [1, -2]$, $b_2 = 0$. Activations: ReLU after layer 1.

For x = (1, 0):

- $z_1 = W_1 \cdot x + b_1 = (1\cdot1 + 1\cdot0 + 0,\ 1\cdot1 + 1\cdot0 + (-1)) = (1, 0)$
- $a_1 = \text{ReLU}(z_1) = (1, 0)$
- $z_2 = W_2 \cdot a_1 + b_2 = 1\cdot1 + (-2)\cdot0 + 0 = 1$
- $y = z_2 = 1$ ✓ (XOR(1, 0) = 1)

Sanity check the other inputs: x = (0, 0) → $a_1 = (0, 0)$, y = 0 ✓; x = (1, 1) → $z_1 = (2, 1)$, $a_1 = (2, 1)$, $y = 2 - 2 = 0$ ✓; x = (0, 1) → $z_1 = (1, 0)$, $a_1 = (1, 0)$, y = 1 ✓.

> **Bottom line:** XOR is THE classical demonstration that single perceptrons cannot represent non-linearly-separable functions. Adding ONE hidden layer with ≥ 2 units fixes it — this is the moment depth becomes mathematically necessary, not just convenient.

Problem 7.2 — Width vs depth at fixed parameter budget

A colleague proposes two networks for the same task, both with roughly 1M parameters:

- Network A: 1 hidden layer with 10,000 ReLU neurons.
- Network B: 30 hidden layers with 100 ReLU neurons each.

Both have the universal-approximation guarantee on paper. The colleague thinks A is simpler and should be preferred.

(a) What does Universal Approximation Theorem actually guarantee, and what does it NOT guarantee?

(b) Why does B typically generalise better on real-world tasks (vision, language)?

(c) Where would A actually be a sensible choice?

Solution — Problem 7.2

(a) UAT — what it says and does not say

UAT: a 1-hidden-layer MLP with enough neurons can REPRESENT any continuous function on a bounded domain to arbitrary precision. It does NOT say: (i) you can find such weights in a reasonable amount of training, (ii) the trained model will generalise to held-out data, (iii) the result will be efficient at inference. UAT is about existence, not about learnability.

(b) Why depth usually wins

Real-world inputs have HIERARCHICAL structure. In vision: pixels → edges → textures → parts → objects. Deep networks naturally model this hierarchy — layer 1 learns edges, layer 5 learns parts, layer 30 learns object identity. A 1-layer 10,000-unit network must reinvent each hierarchical level inside a single layer, which is expressively possible but extraordinarily hard to learn from data.

(c) When width wins

> **Bottom line:** On tabular data with NO hierarchical structure (independent features, no spatial / sequential / linguistic relationship), a wide shallow network can match a deep one and is sometimes easier to train and interpret. UAT is also useful as a theoretical reassurance when designing very-low-depth special-purpose layers (e.g., RBF kernels). For images, text, audio — choose depth.

Problem 7.3 — Forward-pass calculation

Compute the forward pass of the following 2-2-1 sigmoid MLP. Inputs x = (1.0, 2.0). Weights and biases:

- $W_1 = [[0.5, -0.3], [0.2, 0.8]]$
- $b_1 = [0.1, -0.2]$
- $W_2 = [0.6, 0.4]$
- $b_2 = 0.05$

All activations are sigmoid: σ(z) = 1 / (1 + e^(−z)).

(a) Compute the pre-activations z_1 and the activations a_1 of the hidden layer.

(b) Compute the output z_2 and $\hat{y}$.

(c) If the true label is y = 0.7, compute the squared-error loss $L = \frac{1}{2}(y - \hat{y})^2$.

Solution — Problem 7.3

(a) Hidden layer

$z_{11} = 0.5 \cdot 1.0 + (-0.3) \cdot 2.0 + 0.1 = 0.5 - 0.6 + 0.1 = 0.0$

$z_{12} = 0.2 \cdot 1.0 + 0.8 \cdot 2.0 + (-0.2) = 0.2 + 1.6 - 0.2 = 1.6$

$a_{11} = \sigma(0.0) = 0.5$

$a_{12} = \sigma(1.6) = 1 / (1 + e^{(-1.6)}) = 1 / (1 + 0.2019) \approx 0.8320$

(b) Output

$z_2 = 0.6 \cdot 0.5 + 0.4 \cdot 0.8320 + 0.05 = 0.3 + 0.3328 + 0.05 = 0.6828$

$\hat{y} = \sigma(0.6828) = 1 / (1 + e^{(-0.6828)}) = 1 / (1 + 0.5054) \approx 0.6644$

(c) Squared-error loss

Bottom line: $L = \frac{1}{2} \cdot (0.7 - 0.6644)^2 = \frac{1}{2} \cdot (0.0356)^2 \approx \frac{1}{2} \cdot 0.001267 \approx 6.33 \times 10^{-4}$. The prediction is close to the target — a small gradient step in the right direction would close the gap.

Topic 8: Activations and the Vanishing-Gradient Family

Problem 8.1 — Dying ReLU

After training a 20-layer ReLU network with SGD and a moderately large learning rate, a teammate observes that 40% of the hidden neurons in layer 10 always output 0 on ALL validation inputs. They are “dead”.

(a) Walk through the mechanism: how does a ReLU neuron die during training?

(b) Why is this irreversible under vanilla gradient descent?

(c) Name two activation variants designed specifically to prevent or mitigate dying ReLU.

Solution — Problem 8.1

(a) Mechanism

ReLU outputs max(0, z). If a weight update causes the pre-activation z to be negative on every training input, the neuron always outputs 0. The gradient $\partial \text{ReLU}/\partial z = 0$ for $z \leq 0$, so the gradient at THIS neuron’s weights is also 0, and SGD cannot update those weights. The neuron is frozen at output 0 — and stays there for the rest of training.

(b) Why irreversible

Because the gradient is 0, there is no signal to push the weights BACK into a regime where some input would produce $z > 0$. The neuron is stuck below the threshold forever. Vanilla SGD has no mechanism to escape this.

(c) Variants that prevent it

- Leaky ReLU: f(z) = max(0.01z, z). The negative branch has a tiny non-zero slope, so the gradient is always non-zero. A “dying” neuron can recover.
- GELU / Swish: smooth approximations of ReLU that never have a strictly zero gradient anywhere; widely used in modern architectures (BERT, GPT).

Bottom line: Dying ReLU is a real failure mode, exacerbated by large learning rates. Detect it (count zero-activation neurons on a validation batch); prevent it with Leaky ReLU / GELU / careful init + small lr.

Problem 8.2 — Tanh, sigmoid, ReLU at the same x

Three teammates are debating which activation function to use. For each of the following pre-activation values, compute σ(z), tanh(z), and ReLU(z), and also the derivative of each at that value:

- z = −2.0
- z = 0.0
- z = 2.0

Briefly explain what the derivative pattern says about gradient flow through deep networks.

Solution — Problem 8.2

Values

- z = −2.0: σ ≈ 0.1192, tanh ≈ −0.9640, ReLU = 0
- z = 0.0: σ = 0.5, tanh = 0, ReLU = 0
- z = 2.0: σ ≈ 0.8808, tanh ≈ 0.9640, ReLU = 2.0

Derivatives

- σ'(z) = σ(z)·(1−σ(z)): at −2 ≈ 0.105, at 0 = 0.25 (max), at 2 ≈ 0.105
- tanh'(z) = 1 − $\tanh^2(z)$: at −2 ≈ 0.070, at 0 = 1.0 (max), at 2 ≈ 0.070
- ReLU'(z): 0 if z < 0; 1 if z > 0; undefined at z = 0 (typically 0 by convention).

Implication for deep nets

> **Bottom line:** Sigmoid's derivative is bounded above by 0.25; through L layers the gradient shrinks by 0.25^L (vanishing). Tanh's derivative peaks at 1.0 — better than sigmoid but still saturates to 0 at extreme inputs. ReLU's derivative is 1 in the active region, so gradients pass through unchanged — this is the main reason ReLU enabled deep networks.

Problem 8.3 — Vanishing gradient quantified

A 10-layer tanh network is being trained. Suppose each layer has, on average, a pre-activation magnitude such that tanh'(z) ≈ 0.5 in early training.

(a) Estimate the magnitude of the gradient reaching layer 1, relative to the gradient at the output layer.

(b) Now suppose late in training, weights have grown such that tanh saturates often, with tanh'(z) ≈ 0.05. Estimate the new layer-1 gradient ratio.

(c) What does this say about training dynamics over time?

Solution — Problem 8.3

(a) Early training

$\partial L/\partial w_{layer1} \propto (0.5)^{10} \approx 9.77 \times 10^{-4}$ · gradient_output. The early-layer gradient is ~0.001 of the output gradient. Slow but trainable.

(b) Late training (saturated)

$\partial L/\partial w_{layer1} \propto (0.05)^{10} \approx 9.77 \times 10^{-14}$ · gradient_output. Effectively zero — layer 1 stops learning entirely.

(c) Implication

> **Bottom line:** Vanishing gradient is not just an initialisation issue — it can develop OVER TRAINING as weights drift into saturation regimes. Early-stopping, batch normalisation, and residual connections all help by keeping activations in healthy ranges throughout training, not just at start.

Topic 9: Regularisation

Problem 9.1 — L1 sparsity vs L2 smoothness

A team trains a linear model with 50 features. They report:

- With L2 regularisation (λ = 0.1): all 50 weights are small (|w| < 0.5), none exactly zero.
- With L1 regularisation (λ = 0.1): 42 weights are exactly 0; only 8 are non-zero.

(a) Geometrically, why does L1 produce sparsity but L2 does not?

(b) Under which scenario would you prefer L1 over L2?

(c) Briefly describe ElasticNet and when it is useful.

Solution — Problem 9.1

(a) Geometric intuition

L2 penalises Σw^2, a circle in 2-D — its level sets are circles. The optimum lies where the loss contour just touches the circle, typically at a point with both coordinates non-zero. L1 penalises $\Sigma|w|$, a diamond — its level sets are squares rotated 45°. The corners of the diamond lie ON the axes ($w_1 = 0$ or $w_2 = 0$), so the loss contour usually first touches the diamond AT A CORNER, driving one coordinate to exactly zero.

(b) When to prefer L1

- When you suspect many features are irrelevant and want automatic feature selection.
- When interpretability of the final model is important (few non-zero coefficients are easier to explain).
- When inference cost matters (sparse models are faster to evaluate).

(c) ElasticNet

> **Bottom line:** ElasticNet combines L1 + L2: penalty = α · L1 + (1 − α) · L2. The L1 part drives sparsity; the L2 part stabilises among correlated features (L1 alone tends to arbitrarily pick one of a correlated group and zero the rest). ElasticNet is the workhorse for high-dimensional linear models with correlated predictors (genomics, NLP bag-of-words).

Problem 9.2 — Dropout in production

A team trained a network with Dropout p=0.5 on each hidden layer. They deploy it to production and apply Dropout AT INFERENCE TIME. Inference outputs are very noisy — the same input gives slightly different predictions each time.

(a) What is the correct way to use Dropout at inference time?

(b) Explain the “inverted dropout” trick that many libraries use, and why it makes inference straightforward.

(c) When IS noisy inference deliberate, and what is it called?

Solution — Problem 9.2

(a) Correct inference usage

At inference, you turn Dropout OFF — use all neurons. Each neuron’s output must be scaled by p_keep so the EXPECTED output matches what the next layer saw during training. With p=0.5 (drop 50%), each surviving neuron at training time produces an expected output of 0.5 · activation; at inference with no dropout, scale activations by 0.5 to match.

(b) Inverted dropout

Modern libraries (PyTorch, TensorFlow) implement INVERTED dropout: at training, scale the surviving activations UP by 1/p_keep (e.g., 2× when p_keep=0.5). Then at inference, just turn dropout off — no rescaling needed. The expected activation during training already matches the no-dropout output.

(c) Deliberate noisy inference

Bottom line: MC Dropout (Monte Carlo Dropout): keep dropout ON at inference, run K forward passes, average the outputs. The variance across passes is an UNCERTAINTY estimate for the model's prediction. Useful in safety-critical applications (medical imaging, autonomous driving) where you want to know how confident the model is.

Problem 9.3 — Early stopping with patience

A team trains a model with EarlyStopping patience = 5. Training stops at epoch 35; validation loss last improved at epoch 30. A junior asks: "But training loss was still going DOWN at epoch 35 — couldn't we have trained more and gotten a better model?"

(a) Explain in one paragraph why falling training loss alone does NOT justify continuing training.

(b) Under what circumstance would extending training (despite no validation improvement) be defensible?

(c) Suggest TWO alternative regularisation strategies that work alongside early stopping.

Solution — Problem 9.3

(a) Why training loss is not the right signal

Training loss can keep decreasing indefinitely as the model memorises the training set. What ACTUALLY matters is the model's performance on unseen data — proxied by the validation loss. Early stopping says: "I've seen 5 consecutive epochs where validation loss did not improve. The model is now over-fitting (improving on training at the cost of validation). The best version of the model was 5 epochs ago."

(b) When to override

If you have reason to believe the validation set is too small or noisy and shows misleading plateaus, you might increase patience or rely on a separate held-out set. Also, if you are using a STRONG regulariser (heavy dropout, weight decay) and the gap between training and validation is small, you can train longer with less risk of over-fitting.

(c) Complementary regularisation

- Weight decay (L2): adds $\lambda \cdot ||w||^2$ to the loss; encourages small weights throughout training.
- Data augmentation: synthesise more training examples by transforming existing ones (crops, flips, MixUp, etc.). Acts as implicit regularisation.

Bottom line: Early stopping is one knob. Combine with weight decay and data augmentation; the three together are the gold-standard regularisation recipe for modern deep learning.

Topic 10: Convolutional Neural Networks

Problem 10.1 — Output-shape calculator

A medical imaging team is designing a CNN for 256 × 256 grayscale chest X-rays. Compute the spatial output size after each of the following layers (input 256 × 256):

- (i) Conv2D(filters=32, kernel=3, stride=1, padding=1)
- (ii) MaxPool(kernel=2, stride=2)
- (iii) Conv2D(filters=64, kernel=5, stride=1, padding=2)
- (iv) Conv2D(filters=64, kernel=3, stride=2, padding=1)
- (v) MaxPool(kernel=2, stride=2)

(b) Total number of trainable parameters in layer (i), assuming bias terms?

(c) Why might the team choose a stride-2 conv instead of a separate pool layer?

Solution — Problem 10.1

(a) Spatial dimensions

Formula: $O = \lfloor (W - K + 2P) / S \rfloor + 1$.

- (i) (256 − 3 + 2·1) / 1 + 1 = 256. Output 256 × 256 × 32.
- (ii) (256 − 2) / 2 + 1 = 128. Output 128 × 128 × 32.
- (iii) (128 − 5 + 2·2) / 1 + 1 = 128. Output 128 × 128 × 64.
- (iv) (128 − 3 + 2·1) / 2 + 1 = 64. Output 64 × 64 × 64.
- (v) (64 − 2) / 2 + 1 = 32. Output 32 × 32 × 64.

(b) Parameter count of layer (i)

Conv2D parameters = K · K · in_channels · out_channels + out_channels (biases). Layer (i): 3 · 3 · 1 · 32 + 32 = 288 + 32 = 320 parameters.

(c) Stride-2 conv vs pool

> **Bottom line:** A stride-2 conv simultaneously down-samples AND learns features. Max-pool only down-samples (parameter-free). Modern architectures (ResNet, EfficientNet) often prefer stride-2 conv for the first down-sampling step because it gives the network more representational flexibility, at the cost of a few extra parameters.

Problem 10.2 — Parameter explosion at the flatten layer

A CNN's last convolutional output has shape (batch, 14, 14, 256). The team flattens this and passes it through a Dense layer with 4,096 units.

(a) Compute the number of parameters in this single Dense layer.

(b) Why is this "the most parameter-heavy layer" in many traditional CNNs?

(c) What is the modern standard fix, and why does it work without losing accuracy?

Solution — Problem 10.2

(a) Parameter count

Flatten size = 14 · 14 · 256 = 50,176. Dense layer with 4,096 units has 50,176 · 4,096 + 4,096 ≈ 205.6 million parameters in a SINGLE LAYER.

(b) Why this dominates

Convolutional layers are PARAMETER-EFFICIENT: a 3 × 3 kernel with 256 input channels and 256 output channels has only 3·3·256·256 ≈ 590k params. But the flatten-to-dense transition multiplies HUGE spatial × channel sizes by the dense width. In VGG-16, the fully connected layers contain ~120M of the network's ~138M parameters.

(c) Global Average Pooling

> **Bottom line:** Replace the flatten step with Global Average Pooling: average each 14 × 14 channel into a single number, producing a 256-vector. Then a final Dense(num_classes) has only 256 · num_classes + num_classes parameters. ResNet, Inception, EfficientNet all use this. Empirically: no accuracy loss, sometimes BETTER generalisation (the network must learn that each channel is a class-relevant feature).

Problem 10.3 — Fine-tuning a pretrained CNN

A team has a CNN pretrained on ImageNet (1.2M natural images). They want to fine-tune it for satellite imagery — about 5,000 labelled satellite images. They are debating three strategies:

- (A) Freeze ALL conv layers, train only the final Dense classifier.
- (B) Train the entire network end-to-end from the pretrained weights.
- (C) Freeze the EARLY conv layers, fine-tune the LATER conv layers + Dense classifier.

Discuss the trade-offs and recommend one.

Solution — Problem 10.3

(A) Freeze everything

Fastest, lowest risk of overfitting (only the final classifier is learnable, with few params). Works well if satellite images look "somewhat like" ImageNet (have similar edges, textures, low-level structure). May leave performance on the table if satellite images have features that ImageNet never captured.

(B) End-to-end

Maximum flexibility — every weight can adapt. BUT with only 5,000 satellite images and ~25M parameters, you will over-fit catastrophically. The pretrained features will be DESTROYED before useful adaptation happens. Only viable with extensive augmentation, careful learning-rate schedule (very small lr, possibly differential lr per layer), and aggressive regularisation.

(C) Freeze early, train late

Best of both worlds: early layers (edges, textures — TRANSFERABLE to satellite) stay fixed; later layers (high-level features like "dog face" — NOT transferable) are re-trained for satellite-specific concepts ("crop field", "runway"). Standard recipe in transfer learning.

Recommendation

> **Bottom line:** Strategy C. Specifically: freeze the first 2/3 of the conv layers, train the last 1/3 of conv layers + the new classifier with a SMALL learning rate (1e-4 or smaller). After convergence, optionally unfreeze a few more layers and fine-tune further with an even smaller learning rate. This is what nearly every applied transfer-learning paper recommends for ~10^3–10^4 target images.

Topic 11: RNN, LSTM, and Attention

Problem 11.1 — Why vanilla RNNs forget

A team trains a vanilla RNN to predict the next character in long text sequences (200+ characters). After training, the model can repeat words from the last 10–15 characters but completely forgets earlier context. Adding more units (from 100 to 1000) does not help.

(a) What is the mechanism by which vanilla RNNs forget long-range dependencies? Refer to the recurrence equation.

(b) Why does adding more units NOT fix it?

(c) What architectural innovation explicitly solves this, and what is the key component it introduces?

Solution — Problem 11.1

(a) Why RNNs forget

Vanilla RNN: h_t = tanh(W_xh · x_t + W_hh · h_{t−1} + b). The gradient ∂h_t / ∂h_1 = product over t−1 terms involving W_hh and tanh'. Each term is ≤ 1 (tanh' ≤ 1, and W_hh's spectral radius typically ≤ 1 for stable training). Multiplied over 100+ timesteps, the gradient that reaches h_1 is astronomically small. The model literally CANNOT learn to associate something at timestep 200 with something at timestep 1.

(b) Why more units does not help

The vanishing gradient is a property of the RECURRENCE, not the layer width. Adding width gives more capacity at each timestep but does not change the multiplicative chain that destroys long-range gradients.

(c) LSTM and the cell state

> **Bottom line:** LSTM introduces a CELL STATE c_t that flows through time with only ELEMENT-WISE multiplications by gates — not full matrix multiplications. The forget gate decides what to retain, the input gate decides what to add, and the cell state propagates information across hundreds of timesteps without vanishing. GRU is a streamlined version with the same idea. Both make long-range dependencies actually learnable, where vanilla RNN cannot.

Problem 11.2 — LSTM parameter count

An LSTM language model uses input size 200 (word embedding dimension) and hidden size 512.

(a) Compute the number of parameters in ONE LSTM cell (one layer).

(b) If the vocabulary is 50,000 words and the embedding is also 200-dim, how many parameters does the embedding alone have?

(c) If the output layer is a Dense(50,000) (one logit per vocab word), how many params is that?

(d) Total parameters for a 1-layer LSTM language model?

Solution — Problem 11.2

(a) LSTM parameters

LSTM has 4 gates (input i, forget f, candidate g, output o). Each gate has a weight matrix combining the input and the previous hidden state, plus a bias:

per gate: (input + hidden) · hidden + hidden = (200 + 512) · 512 + 512 = 364,544

Total: 4 × 364,544 = 1,458,176 parameters per LSTM layer.

(b) Embedding

Embedding matrix: vocab × embedding_dim = 50,000 · 200 = 10,000,000 parameters.

(c) Output projection

Dense(50,000): hidden · vocab + vocab = 512 · 50,000 + 50,000 = 25,650,000 parameters.

(d) Total

> **Bottom line:** Embedding (10M) + LSTM (1.46M) + Output (25.65M) ≈ 37.1 M parameters. Most of the model is the input and output vocab projections, NOT the recurrent core. Modern models often tie embedding and output weights to halve the vocab params.

Problem 11.3 — Attention by hand

Compute one step of scaled dot-product attention by hand. The query is a single vector; there are 3 keys/values. d_k = 2.

- Q = [1, 0]
- K = [[1, 0], [0, 1], [1, 1]]
- V = [[2, 0], [0, 2], [1, 1]]

(a) Compute the scaled scores K · Q / √(d_k).

(b) Apply softmax to obtain attention weights.

(c) Compute the output as weights · V.

(d) Briefly explain why attention does not have the vanishing-gradient problem RNNs have.

Solution — Problem 11.3

(a) Scores

K · Q = [1·1 + 0·0, 0·1 + 1·0, 1·1 + 1·0] = [1, 0, 1].

Divide by √(d_k) = √2 ≈ 1.414: scaled scores = [0.7071, 0, 0.7071].

(b) Softmax weights

exp(scores) = [exp(0.7071), exp(0), exp(0.7071)] ≈ [2.0281, 1.0000, 2.0281].

Sum = 5.0562. Weights = [0.4011, 0.1978, 0.4011].

(c) Output

output = 0.4011 · [2, 0] + 0.1978 · [0, 2] + 0.4011 · [1, 1]

= [0.8022, 0] + [0, 0.3956] + [0.4011, 0.4011]

= [1.2033, 0.7967].

(d) Why no vanishing gradient

> **Bottom line:** Attention computes the output as a DIRECT WEIGHTED SUM of all values — there is no recursive chain to multiply gradients through. The gradient from the output to ANY of the value vectors is one matrix multiply away, not a 100-step recurrence. This is fundamentally why Transformers (built entirely from attention) train so much better than RNNs on long sequences.

Topic 12: Fairness and Interpretability

Problem 12.1 — Statistical parity vs disparate impact

A bank's loan-approval model produces the following acceptance rates over 1,000 applicants:

- Group A: 600 applicants, 360 approved
- Group B: 400 applicants, 120 approved

(a) Compute P(approve | A) and P(approve | B).

(b) Compute the statistical parity difference: P(approve|A) − P(approve|B).

(c) Compute the disparate impact ratio: P(approve|B) / P(approve|A). Does this satisfy the "80% rule" (EEOC)?

(d) What ADDITIONAL information would you need before concluding this model is "unfair"?

Solution — Problem 12.1

(a) Group rates

P(approve | A) = 360 / 600 = 0.60. P(approve | B) = 120 / 400 = 0.30.

(b) Statistical parity difference

SPD = 0.60 − 0.30 = 0.30. (A 30 percentage-point gap.)

(c) Disparate impact

DI = 0.30 / 0.60 = 0.50. The 80% rule says DI should be ≥ 0.80 for parity. DI = 0.50 fails the rule by a wide margin.

(d) What you still need to know

> **Bottom line:** Statistical parity alone does not establish unfairness — it only flags disparity. You need: (i) the base rates of qualified applicants in each group (if group A genuinely has higher repayment ability, some disparity may be statistically justified); (ii) error rates per group (false-positive and false-negative rates — equalised odds); (iii) the causal story (is the disparity driven by features that are themselves proxies for protected attributes?). Fairness is multi-faceted; one metric is rarely the full answer.

Problem 12.2 — Equalised odds for cancer screening

A diagnostic model for cancer screening has these per-group statistics over 200 patients of each group:

- Group A: 60 actual cancers, 50 caught (TP), 5 false alarms (FP) on 140 healthy
- Group B: 50 actual cancers, 30 caught (TP), 8 false alarms (FP) on 150 healthy

(a) Compute the TPR (sensitivity) for each group.

(b) Compute the FPR for each group.

(c) Does the model satisfy "equalised odds"? Recommend whether it is deployable in a real hospital setting.

Solution — Problem 12.2

(a) TPR per group

- TPR_A = TP / (TP + FN) = 50 / 60 ≈ 0.8333. So 83% of group A's real cancers are caught.
- TPR_B = 30 / 50 = 0.6000. Only 60% of group B's cancers are caught.

Gap: |TPR_A − TPR_B| ≈ 0.2333.

(b) FPR per group

- FPR_A = FP / (FP + TN) = 5 / 140 ≈ 0.0357 (3.6%).
- FPR_B = 8 / 150 ≈ 0.0533 (5.3%).

Gap: |FPR_A − FPR_B| ≈ 0.0176.

(c) Equalised odds and deployment

> **Bottom line:** Equalised odds requires both TPR and FPR to match across groups. Here the FPR gap is small (~1.8%) — acceptable. But the TPR gap is enormous (~23%). Group B is being failed catastrophically: their cancers are 40% less likely to be detected. This model is NOT fit for deployment in its current form. Required mitigations: investigate why TPR diverges (is training data imbalanced? are different risk factors at play?), recalibrate the decision threshold per group OR retrain with a fairness-aware objective (e.g., adversarial debiasing). Do not deploy until TPR is within ≤ 5 percentage points across groups.

Problem 12.3 — SHAP additive decomposition

A credit-scoring model assigns a probability of default to a specific applicant. The SHAP explanation for this prediction is:

- Baseline (average prediction): 0.30
- Income: −0.12 (high income lowers default probability)

- Credit history length: −0.05 (long history lowers default)
- Recent defaults: +0.25 (presence of recent defaults raises probability)
- Debt-to-income: +0.08

(a) Use SHAP's additive property to compute the final prediction f(x). Verify it sums correctly.

(b) Which feature is the strongest reason for THIS applicant's rejection?

(c) Suggest ONE counterfactual recommendation the loan officer could give the applicant to lower their default probability below 0.40.

Solution — Problem 12.3

(a) Verify additivity

$f(x) = \text{baseline} + \Sigma \varphi_i$

$= 0.30 + (-0.12) + (-0.05) + 0.25 + 0.08$

$= 0.30 - 0.12 - 0.05 + 0.25 + 0.08 = 0.46$

Predicted default probability for this applicant: 0.46 (above the 0.30 baseline).

(b) Strongest contributor

The largest absolute contribution is recent_defaults with +0.25. This single feature accounts for more than the rejection above baseline (0.46 − 0.30 = 0.16, but recent_defaults alone added 0.25 in the up direction). It is the single dominant reason for the high predicted risk.

(c) Counterfactual

> **Bottom line:** If recent_defaults dropped to its baseline contribution of 0 (no recent defaults), the predicted probability would fall to 0.46 − 0.25 = 0.21, well below the 0.40 approval threshold. Realistic recommendation: wait until the recent default rolls off (typically 6–7 years in credit reporting). Alternatively, the applicant could increase income or pay down debt to push debt-to-income lower, but these have smaller marginal effects than the recent-default feature. The SHAP attribution makes the path to approval transparent — that is precisely why bank regulators increasingly require SHAP-style explanations for adverse-action notices.

— End of Practice Set —

All the best for your AML examination.